附录 A

ES2018 和 ES2019

从 ECMAScript 2015 开始，TC39 委员会改为每年发布一版新 ECMAScript 规范。这样各个提案可以独立发展，每年所有达到成熟阶段的提案会被打包发布到新一版标准中。不过，打包多少特性并不重要，主要取决于浏览器厂商实现的情况。一旦提案进入第 4 阶段（stage 4），其内容就不会更改，并通常会包含在下一版 ECMAScript 规范中，浏览器就会着手根据自己的计划实现提案的特性。

ECMAScript 2018 于 2018 年 1 月完成，主要增加了异步迭代、剩余和扩展操作符、正则表达式和期约等方面的特性。可以通过 TC39 委员会的 GitHub 仓库了解规范的最新动态。

> **注意**　本附录中介绍的特性相对较新，往往只有新近版本的浏览器才支持。使用这些特性之前，请参考 Can I Use 网站确定支持相应特性的浏览器及其版本。

A.1　异步迭代

在 ECMAScript 最近发布的几个版本中，异步执行和迭代器协议是两个极其热门的主题。异步执行用于释放对执行线程的控制以执行慢操作和收回控制，而迭代器协议则涉及为任意对象定义规范顺序。异步迭代只是这两个概念在逻辑上的统一。

同步迭代器在每次调用 `next()` 时都会返回 `{ value, done }` 对象。当然，这要求确定这个对象内容的计算和资源获取在 `next()` 调用退出时必须完成，否则这些值就无法确定。在使用**同步**迭代器迭代**异步**确定的值时，主执行线程会被阻塞，以等待异步操作完成。

有了异步迭代器，这个问题就迎刃而解了。异步迭代器在每次调用 `next()` 时会提供解决为 `{ value, done }` 对象的期约。这样，执行线程可以释放并在当前这步循环完成之前执行其他任务。

A.1.1　创建并使用异步迭代器

要理解异步迭代器，最简单的办法是用它跟同步迭代器进行比较。下面代码中创建了一个简单的 `Emitter` 类，该类包含一个同步生成器函数，该函数会产生一个同步迭代器，同步迭代器输出 0~4：

```
class Emitter {
  constructor(max) {
    this.max = max;
    this.syncIdx = 0;
  }

  *[Symbol.iterator]() {
    while(this.syncIdx < this.max) {
      yield this.syncIdx++;
```

```
    }
  }
}

const emitter = new Emitter(5);

function syncCount() {
  const syncCounter = emitter[Symbol.iterator]();

  for (const x of syncCounter) {
    console.log(x);
  }
}

syncCount();
// 0
// 1
// 2
// 3
// 4
```

这个例子之所以可以运行起来，主要是因为迭代器可以立即产生下一个值。假如你不想在确定下一个产生的值时阻塞主线程执行，也可以定义异步迭代器函数，让它产生期约包装的值。

为此，要使用迭代器和生成器的异步版本。ECMAScript 2018 为此定义了 `Symbol.asyncIterator`，以便定义和调用输出期约的生成器函数。同时，这一版规范还为异步迭代器增加了 `for-await-of` 循环，用于使用异步迭代器。

相应地，前面的例子可以扩展为同时支持同步和异步迭代：

```
class Emitter {
  constructor(max) {
    this.max = max;
    this.syncIdx = 0;
    this.asyncIdx = 0;
  }

  *[Symbol.iterator]() {
    while(this.syncIdx < this.max) {
      yield this.syncIdx++;
    }
  }

  async *[Symbol.asyncIterator]() {
  // *[Symbol.asyncIterator]() {
    while(this.asyncIdx < this.max) {
      // yield new Promise((resolve) => resolve(this.asyncIdx++));
    yield this.asyncIdx++
    }
  }
}

const emitter = new Emitter(5);

function syncCount() {
  const syncCounter = emitter[Symbol.iterator]();

  for (const x of syncCounter) {
    console.log(x);
```

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

```
  }
}

async function asyncCount() {
  const asyncCounter = emitter[Symbol.asyncIterator]();

  for await(const x of asyncCounter) {
    console.log(x);
  }
}

syncCount();
// 0
// 1
// 2
// 3
// 4

asyncCount();
// 0
// 1
// 2
// 3
// 4
```

为了加深理解，可以把前面例子中的**同步**生成器传给 `for-await-of` 循环：

```
const emitter = new Emitter(5);

async function asyncIteratorSyncCount() {
  const syncCounter = emitter[Symbol.iterator]();

  for await(const x of syncCounter) {
    console.log(x);
  }
}

asyncIteratorSyncCount();
// 0
// 1
// 2
// 3
// 4
```

虽然这里迭代的是同步生成器产生的原始值，但 `for-await-of` 循环仍像它们被包装在期约中一样处理它们。这说明 `for-await-of` 循环可以流畅地处理同步和异步可迭代对象。但是常规 `for` 循环就不能处理异步迭代器了：

```
function syncIteratorAsyncCount() {
  const asyncCounter = emitter[Symbol.asyncIterator]();

  for (const x of asyncCounter) {
    console.log(x);
  }
}

syncIteratorAsyncCount();
// TypeError: asyncCounter is not iterable
```

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

关于异步迭代器，要理解的非常重要的一个概念是 `Symbol.asyncIterator` 符号不会改变生成器函数的行为或者消费生成器的方式。注意在前面的例子中，生成器函数加上了 `async` 修饰符成为异步函数，又加上了星号成为生成器函数。`Symbol.asyncIterator` 在这里只起一个提示的作用，告诉将来消费这个迭代器的外部结构如 `for-await-of` 循环，这个迭代器会返回期约对象的序列。

A.1.2　理解异步迭代器队列

当然，前面的例子是假想的，因为迭代器返回的期约都会立即解决，所以跟同步迭代器的区别很难看出来。想象一下迭代器返回的期约会在不确定的时间解决，而且它们返回的顺序是乱的。异步迭代器应该尽可能模拟同步迭代器，包括每次迭代时代码的按顺序执行。为此，异步迭代器会维护一个回调队列，以保证早期值的迭代器处理程序总是会在处理晚期值之前完成，即使后面的值早于之前的值解决。

为验证这一点，下面的例子中的异步迭代器以随机时长返回期约。异步迭代队列可以保证期约解决的顺序不会干扰迭代顺序。结果应该按顺序打印一组整数（但间隔时间随机）：

```
class Emitter {
  constructor(max) {
    this.max = max;
    this.syncIdx = 0;
    this.asyncIdx = 0;
  }

  *[Symbol.iterator]() {
    while(this.syncIdx < this.max) {
      yield this.syncIdx++;
    }
  }
  async *[Symbol.asyncIterator]() {
    while(this.asyncIdx < this.max) {
      yield new Promise((resolve) => {
        setTimeout(() => {
          resolve(this.asyncIdx++);
        }, Math.floor(Math.random() * 1000));
      });
    }
  }
}

const emitter = new Emitter(5);

function syncCount() {
  const syncCounter = emitter[Symbol.iterator]();

  for (const x of syncCounter) {
    console.log(x);
  }
}

async function asyncCount() {
  const asyncCounter = emitter[Symbol.asyncIterator]();

  for await(const x of asyncCounter) {
    console.log(x);
  }
```

```
}

syncCount();
// 0
// 1
// 2
// 3
// 4

asyncCount();
// 0
// 1
// 2
// 3
// 4
```

A.1.3　处理异步迭代器的 reject()

因为异步迭代器使用期约来包装返回值，所以必须考虑某个期约被拒绝的情况。由于异步迭代会按顺序完成，而在循环中跳过被拒绝的期间是不合理的。因此，被拒绝的期约会强制退出迭代器：

```
class Emitter {
  constructor(max) {
    this.max = max;
    this.asyncIdx = 0;
  }

  async *[Symbol.asyncIterator]() {
    while (this.asyncIdx < this.max) {
      if (this.asyncIdx < 3) {
        yield this.asyncIdx++;
      } else {
        throw 'Exited loop';
      }
    }
  }
}

const emitter = new Emitter(5);

async function asyncCount() {
  const asyncCounter = emitter[Symbol.asyncIterator]();

  for await (const x of asyncCounter) {
    console.log(x);
  }
}

asyncCount();
// 0
// 1
// 2
// Uncaught (in promise) Exited loop
```

A.1.4　使用 next()手动异步迭代

for-await-of 循环提供了两个有用的特性：一是利用异步迭代器队列保证按顺序执行，二是隐藏异步迭代器的期约。不过，使用这个循环会隐藏很多底层行为。

因为异步迭代器仍遵守迭代器协议，所以可以使用 next()逐个遍历异步可迭代对象。如前所述，next()返回的值会包含一个期约，该期约可解决为{ value, done }这样的迭代结果。这意味着必须使用期约 API 获取方法，同时也意味着可以不使用异步迭代器队列。

```
const emitter = new Emitter(5);

const asyncCounter = emitter[Symbol.asyncIterator]();

console.log(asyncCounter.next());
// Promise<{value, done}>
```

A.1.5　顶级异步循环

一般来说，包括 for-await-of 循环在内的异步行为不能出现在异步函数外部。不过，有时候可能确实需要在这样的上下文使用异步行为。为此可以通过创建异步 IIFE 来达到目的：

```
class Emitter {
  constructor(max) {
    this.max = max;
    this.asyncIdx = 0;
  }

  async *[Symbol.asyncIterator]() {
    while(this.asyncIdx < this.max) {
      yield new Promise((resolve) => resolve(this.asyncIdx++));
    }
  }
}

const emitter = new Emitter(5);

(async function() {
  const asyncCounter = emitter[Symbol.asyncIterator]();

  for await(const x of asyncCounter) {
    console.log(x);
  }
})();
// 0
// 1
// 2
// 3
// 4
```

A.1.6　实现可观察对象

异步迭代器可以耐心等待下一次迭代而不会导致计算成本，那么这也为实现可观察对象（Observable）接口提供了可能。总体上看，这涉及捕获事件，将它们封装在期约中，然后把这些事件提供给迭代器，

而处理程序可以利用这些异步迭代器。在某个事件触发时，异步迭代器的下一个期约会解决为该事件。

> **注意**　可观察对象的话题超出了本书范围，因为它们很大程度上是作为第三方库实现的。有兴趣的读者可以了解一下非常流行的 RxJS 库。

下面这个简单的例子会捕获浏览器事件的可观察流。这需要一个期约的队列，每个期约对应一个事件。该队列也会保持事件生成的顺序，对这种问题来说保持顺序也是合理的。

```
class Observable {
  constructor() {
    this.promiseQueue = [];

    // 保存用于解决队列中下一个期约的程序
    this.resolve = null;

    // 把最初的期约推到队列
    // 该期约会解决为第一个观察到的事件
    this.enqueue();
  }

  // 创建新期约，保存其解决方法
  // 再把它保存到队列中
  enqueue() {
    this.promiseQueue.push(
      new Promise((resolve) => this.resolve = resolve));
  }

  // 从队列前端移除期约
  // 并返回它
  dequeue() {
    return this.promiseQueue.shift();
  }
}
```

要利用这个期约队列，可以在这个类上定义一个异步生成器方法。该生成器可用于任何类型的事件：

```
class Observable {
  constructor() {
    this.promiseQueue = [];

    // 保存用于解决队列中下一个期约的程序
    this.resolve = null;

    // 把最初的期约推到队列
    // 该期约会解决为第一个观察到的事件
    this.enqueue();
  }

  // 创建新期约，保存其解决方法
  // 再把它保存到队列中
  enqueue() {
    this.promiseQueue.push(
      new Promise((resolve) => this.resolve = resolve));
  }

  // 从队列前端移除期约
```

```
  // 并返回它
  dequeue() {
    return this.promiseQueue.shift();
  }

  async *fromEvent (element, eventType) {
    // 在有事件生成时，用事件对象来解决队列头部的期约
    // 同时把另一个期约加入队列
    element.addEventListener(eventType, (event) => {
      this.resolve(event);
      this.enqueue();
    });

    // 每次解决队列前面的期约
    // 都会向异步迭代器返回相应的事件对象
    while (1) {
      yield await this.dequeue();
    }
  }
}
```

这样，这个类就定义完了。接下来在 DOM 元素上定义可观察对象就很简单了。假设页面上有一个 `<button>`元素，可以像下面这样捕获该按钮上的一系列 `click` 事件，然后在控制台把它们打印出来：

```
class Observable {
  constructor() {
    this.promiseQueue = [];

    // 保存用于解决队列中下一个期约的程序
    this.resolve = null;

    // 把最初的期约推到队列
    // 该期约会解决为第一个观察到的事件
    this.enqueue();
  }

  // 创建新期约，保存其解决方法
  // 再把它保存到队列中
  enqueue() {
    this.promiseQueue.push(
      new Promise((resolve) => this.resolve = resolve));
  }

  // 从队列前端移除期约
  // 并返回它
  dequeue() {
    return this.promiseQueue.shift();
  }

  async *fromEvent (element, eventType) {
    // 在有事件生成时，用事件对象来解决 队列头部的期约
    // 同时把另一个期约加入队列
    element.addEventListener(eventType, (event) => {
      this.resolve(event);
      this.enqueue();
    });

    // 每次解决队列前面的期约
    // 都会向异步迭代器返回相应的事件对象
```

```
    while (1) {
      yield await this.dequeue();
    }
  }
}

(async function() {
  const observable = new Observable();

  const button = document.querySelector('button');
  const mouseClickIterator = observable.fromEvent(button, 'click');

  for await (const clickEvent of mouseClickIterator) {
    console.log(clickEvent);
  }
})();
```

A.2　对象字面量的剩余操作符和扩展操作符

ECMAScript 2018 将数组中的剩余操作符和扩展操作符也移植到了对象字面量。这极大地方便了对象合并和通过其他对象创建新对象。

A.2.1　剩余操作符

剩余操作符可以在解构对象时将所有剩下未指定的可枚举属性收集到一个对象中。比如：

```
const person = { name: 'Matt', age: 27, job: 'Engineer' };
const { name, ...remainingData } = person;

console.log(name); // Matt
console.log(remainingData); // { age: 27, job: 'Engineer' }
```

每个对象字面量中最多可以使用一次剩余操作符，而且必须放在最后。因为每个对象字面量只能有一个剩余操作符，所以可以嵌套剩余操作符。嵌套时，因为子属性对应的剩余操作符没有歧义，所以返回对象的内容不会重叠：

```
const person = { name: 'Matt', age: 27, job: { title: 'Engineer', level: 10 } };

const { name, job: { title, ...remainingJobData }, ...remainingPersonData } = person;

console.log(name);                  // Matt
console.log(title);                 // Engineer
console.log(remainingPersonData);   // { age: 27 }
console.log(remainingJobData);      // { level: 10 }

const { ...a, job } = person;
// SyntaxError: Rest element must be last element
```

剩余操作符在对象间执行浅复制，因此会复制对象的引用而不会克隆整个对象：

```
const person = { name: 'Matt', age: 27, job: { title: 'Engineer', level: 10 } };

const { ...remainingData } = person;

console.log(person === remainingData);          // false
console.log(person.job === remainingData.job);  // true
```

剩余操作符会复制所有自有可枚举属性，包括符号：

```
const s = Symbol();
const foo = { a: 1, [s]: 2, b: 3 }

const {a, ...remainingData} = foo;

console.log(remainingData);
// { b: 3, Symbol(): 2 }
```

A.2.2　扩展操作符

扩展操作符可以像拼接数组一样合并两个对象。应用到内部对象的扩展操作符会对所有自有可枚举属性执行浅复制到外部对象，包括符号：

```
const s = Symbol();
const foo = { a: 1 };
const bar = { [s]: 2 };

const foobar = {...foo, c: 3, ...bar};

console.log(foobar);
// { a: 1, c: 3 Symbol(): 2 }
```

扩展对象的顺序很重要，主要有两个原因。

(1) 对象跟踪插入顺序。从扩展对象复制的属性按照它们在对象字面量中列出的顺序插入。

(2) 对象会覆盖重名属性。在出现重名属性时，会使用后出现属性的值。

下面的例子演示了上述约定：

```
const foo = { a: 1 };
const bar = { b: 2 };

const foobar = {c: 3, ...bar, ...foo};

console.log(foobar);
// { c: 3, b: 2, a: 1 }

const baz = { c: 4 };

const foobarbaz = {...foo, ...bar, c: 3, ...baz};

console.log(foobarbaz);
// { a: 1, b: 2, c: 4 }
```

与剩余操作符一样，所有复制都是浅复制：

```
const foo = { a: 1 };
const bar = { b: 2, c: { d: 3 } };

const foobar = {...foo, ...bar};

console.log(foobar.c === bar.c); // true
```

A.3　`Promise.prototype.finally()`

以前，只能使用不太好看的方式处理期约退出“待定”状态，而不管后续状态如何。换句话说，通

常需要重复利用处理程序：

```
let resolveA, rejectB;

function finalHandler() {
  console.log('finished');
}

function resolveHandler(val) {
  console.log('resolved');
  finalHandler();
}

function rejectHandler(err) {
  console.log('rejected');
  finalHandler();
}

new Promise((resolve, reject) => {
  resolveA = resolve;
})
.then(resolveHandler, rejectHandler);

new Promise((resolve, reject) => {
  rejectB = reject;
})
.then(resolveHandler, rejectHandler);

resolveA();
rejectB();
// resolved
// finished
// rejected
// finished
```

有了 `Promise.prototype.finally()`，就可以统一共享的处理程序。`finally()`处理程序不传递任何参数，也不知道自己处理的期约是解决的还是拒绝的。前面的代码可以重构为如下形式：

```
let resolveA, rejectB;

function finalHandler() {
  console.log('finished');
}

function resolveHandler(val) {
  console.log('resolved');
}

function rejectHandler(err) {
  console.log('rejected');
}

new Promise((resolve, reject) => {
  resolveA = resolve;
})
.then(resolveHandler, rejectHandler)
.finally(finalHandler);
```

```
new Promise((resolve, reject) => {
  rejectB = reject;
})
.then(resolveHandler, rejectHandler)
.finally(finalHandler);

resolveA();
rejectB();
// resolved
// rejected
// finished
// finished
```

注意日志的顺序不一样了。每个 `finally()`都会创建一个新期约实例，而这个新期约会添加到浏览器的微任务队列，只有前面的处理程序执行完成才会解决。

A.4　正则表达式相关特性

ECMAScript 2018 为正则表达式增加了一些特性。

A.4.1　`dotAll` 标志

正则表达式中用于匹配任意字符的点（`.`）不匹配换行符，比如`\n`和`\r`或非 BMP 字符，如表情符号。

```
const text = `
foo
bar
`;

const re = /foo.bar/;

console.log(re.test(text)); // false
```

为此，规范新增了 `s` 标志（意思是单行，singleline），从而解决了这个问题：

```
const text = `
foo
bar
`;

const re = /foo.bar/s;

console.log(re.test(text)); // true
```

A.4.2　向后查找断言

正则表达式支持肯定式向前查找断言和否定式向前查找断言，可以匹配后跟指定字符串的表达式：

```
const text = 'foobar';

// 肯定式向前查找
// 断言后跟某个值，但不捕获该值
const rePositiveMatch = /foo(?=bar)/;
const rePositiveNoMatch = /foo(?=baz)/;
```

```
console.log(rePositiveMatch.exec(text));
// ["foo"]

console.log(rePositiveNoMatch.exec(text));
// null

// 否定式向前查找
// 断言后面不是某个值，但不捕获该值
const reNegativeNoMatch = /foo(?!bar)/;
const reNegativeMatch = /foo(?!baz)/;

console.log(reNegativeNoMatch.exec(text));
// null

console.log(reNegativeMatch.exec(text));
// ["foo"]
```

规范相应地增加了与这些断言对应的肯定式向后查找和否定式向后查找。向后查找与向前查找的工作原理类似，只是会检测要捕获内容之前的内容。

```
const text = 'foobar';

// 肯定式向后查找
// 断言前面是某个值，但不捕获该值
const rePositiveMatch = /(?<=foo)bar/;
const rePositiveNoMatch = /(?<=baz)bar/;

console.log(rePositiveMatch.exec(text));
// ["bar"]

console.log(rePositiveNoMatch.exec(text));
// null

// 否定式向后查找
// 断言前面不是某个值，但不捕获该值
const reNegativeNoMatch = /(?<!foo)bar/;
const reNegativeMatch = /(?<!baz)bar/;

console.log(reNegativeNoMatch.exec(text));
// null

console.log(reNegativeMatch.exec(text));
// ["bar"]
```

A.4.3　命名捕获组

多个捕获组通常是按索引来取值的，但索引没有上下文，反映不出它们包含的是什么内容：

```
const text = '2018-03-14';

const re = /(\d+)-(\d+)-(\d+)/;

console.log(re.exec(text));
// ["2018-03-14", "2018", "03", "14"]
```

为此，规范支持将捕获组与有效 JavaScript 标识符关联，这样就可以通过标识符获取捕获组的内容：

```
const text = '2018-03-14';

const re = /(?<year>\d+)-(?<month>\d+)-(?<day>\d+)/;

console.log(re.exec(text).groups);
// { year: "2018", month: "03", day: "14" }
```

A.4.4　Unicode 属性转义

Unicode 标准为每个字符都定义了属性。字符属性，涉及字符名称、类别、空格指示和字符内部定义的脚本或语言等。通过使用 Unicode 属性转义可以在正则表达式中使用这些属性。

有些属性是二进制，意味可以独立使用。比如 `Uppercase` 和 `White_Space`。有些属性是键/值对，即一个属性对应一个属性值。比如 `Script_Extensions=Greek`。

Unicode 属性列表及属性值列表参见 Unicode 网站。

Unicode 属性转义在正则表达式中可使用`\p` 表示匹配，使用`\P` 表示不匹配：

```
const pi = String.fromCharCode(0x03C0);
const linereturn = `
`;

const reWhiteSpace = /\p{White_Space}/u;
const reGreek = /\p{Script_Extensions=Greek}/u;
const reNotWhiteSpace = /\P{White_Space}/u;
const reNotGreek = /\P{Script_Extensions=Greek}/u;

console.log(reWhiteSpace.test(pi));              // false
console.log(reWhiteSpace.test(linereturn));      // true
console.log(reNotWhiteSpace.test(pi));           // true
console.log(reNotWhiteSpace.test(linereturn));   // false

console.log(reGreek.test(pi));                   // true
console.log(reGreek.test(linereturn));           // false
console.log(reNotGreek.test(pi));                // false
console.log(reNotGreek.test(linereturn));        // true
```

A.5　数组打平方法

ECMAScript 2019 在 `Array.prototype` 上增加了两个方法：`flat()`和 `flatMap()`。这两个方法为打平数组提供了便利。如果没有这两个方法，则打平数组就要使用迭代或递归。

> **注意**　`flat()`和 `flatMap()`只能用于打平嵌套数组。嵌套的可迭代对象如 `Map` 和 `Set` 不能打平。

A.5.1　`Array.prototype.flatten()`

下面是如果没有这两个新方法要打平数组的一个示例实现：

```
function flatten(sourceArray, flattenedArray = []) {
  for (const element of sourceArray) {
    if (Array.isArray(element)) {
```

```
      flatten(element, flattenedArray);
    } else {
      flattenedArray.push(element);
    }
  }
  return flattenedArray;
}

const arr = [[0], 1, 2, [3, [4, 5]], 6];

console.log(flatten(arr))
// [0, 1, 2, 3, 4, 5, 6]
```

这个例子在很多方面像一个树形数据结构：数组中每个元素都像一个子节点，非数组元素是叶节点。因此，这个例子中的输入数组是一个高度为 2 有 7 个叶节点的树。打平这个数组本质上是对叶节点的按序遍历。

有时候如果能指定打平到第几级嵌套是很有用的。比如下面这个例子，它重写了上面的版本，允许指定要打平几级：

```
function flatten(sourceArray, depth, flattenedArray = []) {
  for (const element of sourceArray) {
    if (Array.isArray(element) && depth > 0) {
      flatten(element, depth - 1, flattenedArray);
    } else {
      flattenedArray.push(element);
    }
  }

  return flattenedArray;
}


const arr = [[0], 1, 2, [3, [4, 5]], 6];

console.log(flatten(arr, 1));
// [0, 1, 2, 3, [4, 5], 6]
```

为了解决上述问题，规范增加了 `Array.prototype.flat()`方法。该方法接收 `depth` 参数（默认值为 1），返回一个对要打平 `Array` 实例的浅复制副本。下面看几个例子：

```
const arr = [[0], 1, 2, [3, [4, 5]], 6];

console.log(arr.flat(2));
// [0, 1, 2, 3, 4, 5, 6]

console.log(arr.flat());
// [0, 1, 2, 3, [4, 5], 6]
```

因为是执行浅复制，所以包含循环引用的数组在被打平时会从源数组复制值：

```
const arr = [[0], 1, 2, [3, [4, 5]], 6];

arr.push(arr);

console.log(arr.flat());
// [0, 1, 2, 3, 4, 5, 6, [0], 1, 2, [3, [4, 5]], 6]
```

A.5.2 Array.prototype.flatMap()

Array.prototype.flatMap()方法会在打平数组之前执行一次映射操作。在功能上，arr.flatMap(f)与 arr.map(f).flat()等价；但 arr.flatMap()更高效，因为浏览器只需要执行一次遍历。

flatMap()的函数签名与 map()相同。下面是一个简单的例子：

```
const arr = [[1], [3], [5]];

console.log(arr.map(([x]) => [x, x + 1]));
// [[1, 2], [3, 4], [5, 6]]

console.log(arr.flatMap(([x]) => [x, x + 1]));
// [1, 2, 3, 4, 5, 6]
```

flatMap()在非数组对象的方法返回数组时特别有用，例如字符串的 split()方法。来看下面的例子，该例子把一组输入字符串分割为单词，然后把这些单词拼接为一个单词数组：

```
const arr = ['Lorem ipsum dolor sit amet,', 'consectetur adipiscing elit.'];

console.log(arr.flatMap((x) => x.split(/[\W+]/)));
// ["Lorem", "ipsum", "dolor", "sit", "amet", "", "consectetur", "adipiscing",
"elit", ""]
```

对于上面的例子，可以利用空数组进一步过滤上一次映射后的结果，这也是一个数据处理技巧（虽然可能会有些性能损失）。下面的例子扩展了上面的例子，去掉了空字符串:

```
const arr = ['Lorem ipsum dolor sit amet,', 'consectetur adipiscing elit.'];

console.log(arr.flatMap((x) => x.split(/[\W+]/)).flatMap((x) => x || []));
// ["Lorem", "ipsum", "dolor", "sit", "amet", consectetur", "adipiscing", "elit"]
```

这里，结果中的每个空字符串首先映射到一个空数组。在打平时，这些空数组就会因为没有内容而被忽略。

> **注意**　不建议使用这个技巧。这是因为过滤每个值都要构建一个立即丢弃的 Array 实例。

A.6 Object.fromEntries()

ECMAScript 2019 又给 Object 类添加了一个静态方法 fromEntries()，用于通过键/值对数组的集合构建对象。这个方法执行与 Object.entries()方法相反的操作。来看下面的例子：

```
const obj = {
  foo: 'bar',
  baz: 'qux'
};

const objEntries = Object.entries(obj);

console.log(objEntries);
// [["foo", "bar"], ["baz", "qux"]]

console.log(Object.fromEntries(objEntries));
// { foo: "bar", baz: "qux" }
```

此静态方法接收一个可迭代对象参数，该可迭代对象可以包含任意数量的大小为 2 的可迭代对象。这个方法可以方便地将 Map 实例转换为 Object 实例，因为 Map 迭代器返回的结果与 fromEntries() 的参数恰好匹配：

```
const map = new Map().set('foo', 'bar');

console.log(Object.fromEntries(map));
// { foo: "bar" }
```

A.7　字符串修理方法

ECMAScript 2019 向 String.prototype 添加了两个新方法：trimStart() 和 trimEnd()。它们分别用于删除字符串开头和末尾的空格。这两个方法旨在取代之前的 trimLeft() 和 trimRight()，因为后两个方法在从右往左书写的语言（如阿拉伯语和希伯来语）中有歧义。

在只有一个空格的情况下，这两个方法相当于执行与 padStart() 和 padEnd() 相反的操作。下面的例子演示了使用这两个方法删除字符串前后的空格：

```
let s = '   foo   ';

console.log(s.trimStart()); // "foo   "
console.log(s.trimEnd());   // "   foo"
```

A.8　Symbol.prototype.description

ECMAScript 2019 在 Symbol.prototype 上增加了 description 属性，用于取得可选的符号描述。以前，只能通过将符号转型为字符串来取得这个描述：

```
const s = Symbol('foo');

console.log(s.toString());
// Symbol(foo)
```

这个原型属性是只读的，可以在实例上直接取得符号的描述。如果没有描述，则默认为 undefined。

```
const s = Symbol('foo');

console.log(s.description);
// foo
```

A.9　可选的 catch 绑定

在 ECMAScript 2019 之前，try/catch 块的结构相当严格。即使不想使用捕获的错误对象，解析器也要求必须在 catch 子句中把该对象赋值给一个变量：

```
try {
  throw 'foo';
} catch (e) {
  // 发生错误了，但你不想使用错误对象
}
```

在 ECMAScript 2019 中，可以省略这个赋值，并完全忽略错误：

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

```
try {
  throw 'foo';
} catch {
  // 发生错误了，但你不想使用错误对象
}
```

A.10　其他新增内容

ES2019 还对现有 API 进行了其他一些调整。

- `Array.prototype.sort()`**稳定**了，意味着相同的对象在输出中不会被重新排序。
- 由于单独的 UTF-16 代理对字符不能使用 UTF-8 编码，`JSON.stringify()` 在 ES2019 以前会返回 UTF-16 码元，现在则改为返回转义序列，也是有效的 Unicode。
- ES2019 以前，U+2028 LINE SEPARATOR 和 U+2029 PARAGRAPH SEPARATOR 在 JSON 字符串中都有效，但在 ECMAScript 字符串中则无效。ES2019 实现了 ECMAScript 字符串与 JSON 字符串的兼容。
- ES2019 以前，浏览器厂商可以自由决定 `Function.prototype.toString()`返回什么。ES2019 要求这个方法尽可能返回函数的源代码，否则返回`{ [native code] }`。

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权